Guion de demo

Quantum Testing Agent — Vultr: Agent Rush Hackathon

Jesús León Porras

8 de noviembre de 2026

Abstract

Guion de demostración para el proyecto Quantum Testing Agent, participante en el desafío Blast Radius Zero del hackathon Vultr: Agent Rush. Duración máxima: 5 minutos. Incluye un momento de contención obligatorio donde el sandbox absorbe código hostil sin comprometer el host ni el backend.

1 Objetivo de la demo

Demostrar un agente web que prueba circuitos cuánticos de forma aislada y verificable en Vultr. La demo debe mostrar:

1. Planificación del agente (LLM de Vultr Serverless Inference).
2. Generación de código en el framework elegido.
3. Ejecución en sandbox Docker aislado con límites.
4. Resultado verificable (estados, histogramas, logs).
5. **Momento de contención:** sandbox absorbe código hostil y se destruye.
6. Compatibilidad multi-framework.

2 Guion paso a paso

3 Momento de contención (clave para el vídeo)

- **Entrada hostil:** código con `while True: pass` o `os.system("rm -rf /tmp/test")`.
- **Comportamiento esperado:** el contenedor alcanza el límite de tiempo/memoria, Docker lo mata, el entorno se destruye, la API responde con `containment: true`.
- **Evidencia en vídeo:** terminal del backend registrando la destrucción del contenedor + web mostrando estado seguro.

4 Requisitos técnicos para la demo

5 Guion de voz (sugerido)

> "Hola, soy el agente de pruebas cuánticas. Este es un circuito de 3 qubits con Hadamard y CNOT. Lo voy a ejecutar de forma aislada en un sandbox de Docker en Vultr. El agente planificó la prueba, generó el código en Qiskit y lo ejecutó dentro del contenedor. Aquí está el resultado verificable: estados de medida y histogramas.

#	Paso	Qué se muestra	Tiempo
1	Apertura	Problema: probar código cuántico es riesgoso; solución: sandbox aislado	30 s
2	Entrada	Usuario escribe circuito de 3 qubits + Hadamard + CNOT en Qiskit	30 s
3	Planificación	Agente descompone en pasos, muestra plan en pantalla	30 s
4	Generación	Código Qiskit generado, mostrado brevemente	20 s
5	Ejecución	Contenedor Docker levantado con límites, código ejecutado	45 s
6	Resultado	Estados de medida y histogramas verificables	30 s
7	Contención	Código hostil (bucle infinito) → sandbox lo absorbe y destruye	60 s
8	Multi-framework	Selección de PennyLane/Cirq, prueba repetida	30 s
9	Cierre	Resumen de arquitectura + llamada a acción	45 s

Table 1: Guion de demo (≈ 4 min 30 s)

Requisito	**Detalle**
Backend desplegado	VM de Vultr con FastAPI + Docker, URL pública
LLM	Vultr Serverless Inference
Frameworks en Docker	Qiskit, PennyLane (Cirq, Azure opcionales)
Límites de sandbox	60 s CPU, 512 MB memoria, red desactivada
Repo público	GitHub con README, setup, docs, guion de demo
Vídeo	$\leq$ 5 min, MP4, $\leq$ 300 MB, con contención

Table 2: Requisitos técnicos

Ahora voy a probar algo peligroso: un bucle infinito embebido en el circuito. Observen: el sandbox lo detecta, alcanza el límite de tiempo, destruye el contenedor y el sistema sigue intacto. Eso es contención.

Todo corre en una VM de Vultr, con llamadas LLM a Vultr Serverless Inference, sin GPUs de Vultr, usando simulador CPU y backends remotos opcionales de IBM, AWS y Azure."

6 Checklist pre-demo

Backend desplegado en VM de Vultr con URL pública accesible.

`.env` configurado con `VULTR_API_KEY`.

Docker disponible en la VM.

Imagen `python:3.11-slim` con Qiskit y PennyLane instalados.

Prueba manual de un circuito simple completada con éxito.

Prueba de contención realizada y registrada.

Repo GitHub público con README y documentación.

Vídeo grabado ($\leq$ 5 min, MP4, $\leq$ 300 MB).

Envío realizado en lablab.ai antes del 8 de noviembre.